

Serverless



CUHK

Motivation

- Under-utilization of “provisioned” model
 - Tenant:
 - Often need to over-subscribe in case of peak usage
 - Don't want to pay for idleness
 - Provider:
 - Often over-subscribe in the back
 - But at the risk of SLA violation penalty
 - Serverless can reduce the granularity → pack better

Viewing 560 of 560 available instances

< 1 2 3 4 5 6 7 ... 28					
Instance name ▲	On-Demand hourly rate ▼	vCPU ▼	Memory ▼	Storage ▼	Network performance
a1.medium	\$0.0255	1	2 GiB	EBS Only	Up to 10 Gigabit
a1.large	\$0.051	2	4 GiB	EBS Only	Up to 10 Gigabit
a1.xlarge	\$0.102	4	8 GiB	EBS Only	Up to 10 Gigabit
a1.2xlarge	\$0.204	8	16 GiB	EBS Only	Up to 10 Gigabit
a1.4xlarge	\$0.408	16	32 GiB	EBS Only	Up to 10 Gigabit

When United Airlines overbooks a flight...



Serverless

- A misnomer
 - Things still run on servers
- Just you are not renting a server/vm
- Payment model is pay-per-use in terms of *GB-second*
 - *1st second: 5GB memory allocated. $\$0.000000166667 * 5$*
 - *+ 2nd sec: 4GB memory allocated. $\$0.000000166667 * 4$*
 - *+ 3rd sec:*
 - *...*
 -

AWS Lambda Pricing

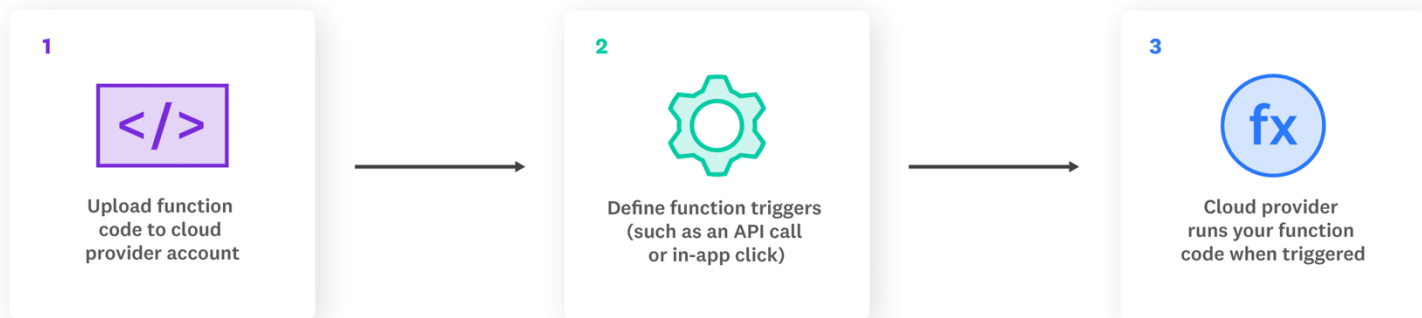
Region: US East (Ohio) ▾

Architecture	Duration	Requests
x86 Price		
First 6 Billion GB-seconds / month	\$0.0000166667 for every GB-second	\$0.20 per 1M requests
Next 9 Billion GB-seconds / month	\$0.000015 for every GB-second	\$0.20 per 1M requests
Over 15 Billion GB-seconds / month	\$0.0000133334 for every GB-second	\$0.20 per 1M requests

First generation

- Function as a service (FaaS)
- Wrap inside containers (binary with all necessary libraries)
- Only for **stateless** functions (why?!!!)

How Serverless Functions Work



Serverless can infinitely scale out...

- Examples use cases:
 - User uploads a 1200 x 800 image, triggers a FaaS to create 300 x 200 thumbnail
 - Simple web request dispatch
- Support the so-called “infinite scale out” only for **embarrassingly parallel** (or read-only) problems

Customers **love** the pay-per-use model

- They then ask
 - “Can we use do normal programming (with state) on serverless platform?”
- From FaaS providers’ point of view, they want:
 - Instances of function `hi` can run on different machines
 - Functions `hi` and `bye` of the same program can run on different machines
- Then, programming languages with ‘global variable’ (shared memory) becomes very tricky: `global int cur_count++`
- So, either they change the system to support ‘global variable’ natively, or they ask you write like this:

```
On event e:  
  counting()
```

```
func counting()  
  cur_count = S3.load(my_counter)  
  cur_count++  
  S3.store(cur_count)
```

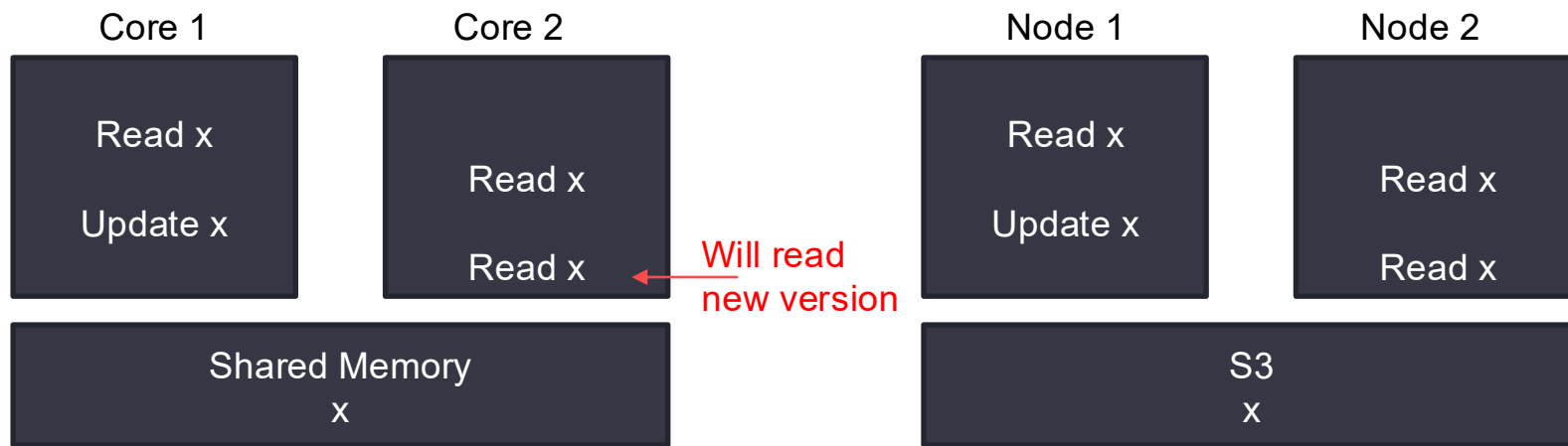
Race conditions!

Multiple function instances in FaaS <-> multi-threading!

```
func counting()  
  cur_count = S3.load(my_counter)  
  cur_count++  
  S3.store(cur_count)
```

```
func counting()  
  cur_count = S3.load(my_counter)  
  cur_count++  
  S3.store(cur_count)
```

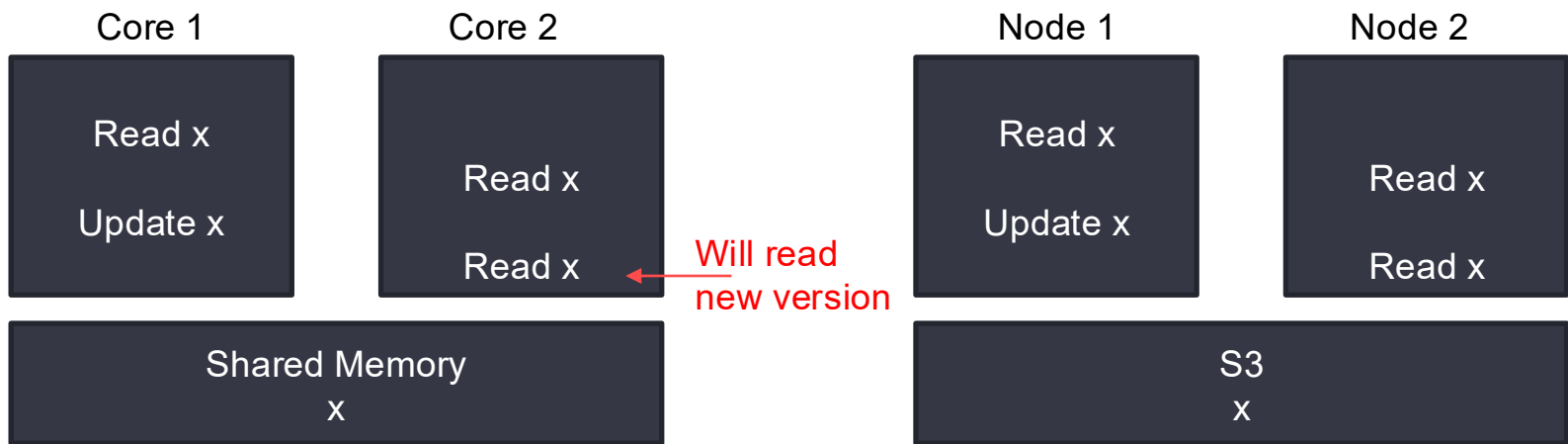
Serverless does not have cache coherence!



What a normal programmer expects: (except ARM programmers)

- shared memory model
 - with hardware **cache coherence**
 - synchronous/reliable
- handling race by themselves

Serverless does not have cache coherence!



Cloud providers have to **simulate** shared memory model with **cache coherence**

- using S3 as the shared memory (or w/o S3)
- under **asynchronous** setting

Options for stateful programming on serverless

Change to DB programming	0 hardware change Let the DB do resolve the conflicting stuff for you
Change to S3 programming	0 hardware change Painful handling of race condition by yourself
Build an OS to abstract a cluster of asynchronous machines as an “x86 (Total-Store-Ordering)” machine	0 hardware change; 0 software change; Happy general user, painful OS programmers
Change the hardware to CXL	Hardware change; 0 software change

- That explains why current FaaS are stateless!
 - Just because they are not ready yet

CXL3.0

Marvell Launches Next-generation CXL Switch, Enabling Memory Pooling to Break Through the AI “Memory Wall”

March 17, 2026

Marvell Structera S Pooling Capability Offers Higher Memory Utilization, Improved Data Flow Efficiency and AI Application Performance, and Greater Infrastructure Flexibility

LOS ANGELES--(BUSINESS WIRE)-- **OFC 2026** – Marvell Technology, Inc. (NASDAQ: MRVL), a leader in data infrastructure semiconductor solutions, today announced Marvell® Structera™ S 30260—a new 260-lane CXL switch device that enables rack-level memory pooling. Working in concert with [Marvell Structera A near-memory accelerators](#), [Structera X memory-expansion controllers](#) and [Alaska® P PCIe/CXL retimers](#), the new switch allows data center operators to further increase memory bandwidth and capacity by providing access to disaggregated memory resources outside the server.

CXL switching has long supported traditional CPU computing architectures and has also now become essential for accelerated AI data center infrastructure. As AI clusters expand dramatically in scale and complexity, memory capacity has hit a wall, becoming a primary constraint on the performance and efficiency of these systems. Exploding large language model (LLM) sizes, expanding context windows and growing key-value cache requirements are driving unprecedented memory demands. Traditional approaches are becoming increasingly inefficient and costly, and they cannot support the massive capacity required by these larger and more complex AI workloads.

Leveraging industry-leading switch solutions through the recent acquisition of XConn Technologies, the Structera S CXL switch device enables true memory pooling across the rack, allowing hyperscalers and data center operators to dynamically expand and allocate memory resources across CPUs, GPUs, XPU and other accelerators without replacing existing platforms or relying solely on HBM stacking. It enables a large scalable, composable resource that can deliver higher memory utilization, improved data flow efficiency and AI application performance, greater infrastructure design flexibility and scalability, and reduced overall total cost of ownership.